

SPLoRA: Sonnet Generation and Paraphrase Detection with LoRA

Stanford CS224N Default Project

Joseph Bailey

Department of Computer Science
Stanford University
josephrb@stanford.edu

Maya Vendhan

Department of Computer Science
Stanford University
mvendhan@stanford.edu

Abstract

We investigate Low-Rank Adaptation (LoRA) as a parameter-efficient alternative to full fine-tuning of GPT-2, evaluating it on two distinct tasks: paraphrase detection (Quora Question Pairs) and Shakespearean sonnet generation. We systematically study adapter placement—attention-only, MLP-only, and combined—alongside the effects of rank, scaling factor, learning rate, and dropout. Across both tasks, adapting both MLP and attention layers consistently outperforms single-module configurations. For paraphrase detection, our best LoRA configuration achieves 0.888 development accuracy—within 0.6% of full fine-tuning—while training only ~1% of the model’s 124M parameters. For sonnet generation, our best configuration reaches a chrF of 41.94, surpassing the full fine-tuning baseline of 40.417. Notably, the optimal hyperparameters differ across tasks: classification favors moderate scaling ($\alpha = 16$) with no dropout, while generation benefits from stronger scaling ($\alpha = 32$) and moderate dropout ($p = 0.2$), suggesting that LoRA configurations should be tuned per-task rather than treated as universal.

1 Key Information to include

- Mentor: Advit Deepak
- External Collaborators (if you have any): None
- Sharing project: No
- Late Days (if you are using any):

Team member	Late days remaining
Joseph Bailey	4
Maya Vendhan	5
Total pooled	9
Team size	2
Late days applied for the report	1

2 Introduction

Large pretrained language models have achieved strong performance across a wide range of natural language processing tasks, but adapting these models to new tasks typically requires fine-tuning hundreds of millions or billions of parameters. This process can be computationally expensive and memory-intensive, making it impractical for many settings. Parameter-efficient fine-tuning methods aim to address this challenge by adapting models while updating only a small subset of parameters. In this work we investigate Low-Rank Adaptation (LoRA) as a method for efficiently adapting GPT-2 to two tasks: paraphrase detection on the Quora Question Pairs (QQP) dataset and stylistic text generation using Shakespearean sonnets.

Efficient model adaptation is increasingly important as model sizes continue to grow. Even relatively small transformer models such as GPT-2 contain hundreds of millions of parameters, and modern models are orders of magnitude larger. Reducing the number of trainable parameters can therefore significantly lower training cost, memory usage, and storage requirements, enabling faster experimentation and broader deployment. However, it remains unclear how parameter-efficient methods behave across different task types, particularly when comparing structured classification tasks and creative generative tasks.

This raises a central question: does a single LoRA configuration perform well across fundamentally different tasks, or must adapter design choices be tuned per-task? Classification tasks like paraphrase detection require learning subtle semantic distinctions in a fixed output space, while generative tasks like sonnet writing require capturing stylistic and structural properties of an open-ended output distribution. If optimal configurations diverge between these regimes, it would suggest that practitioners cannot simply adopt a default LoRA recipe.

In the case of LoRA, performance depends on several design choices, including which transformer modules are adapted, the rank of the low-rank update, and scaling and regularization hyperparameters, such as the LoRA scaling factor α and dropout. Poorly chosen configurations can lead to underfitting, overfitting, or ineffective parameter updates, and the optimal settings may vary substantially across tasks. A configuration that performs well for one task like classification may not generalize to another task like generative modeling, emphasizing the need for systematic evaluation of configuration design.

In this project we implement LoRA directly within our GPT-2 model and systematically evaluate several LoRA design choices. We compare adapter placement strategies including attention-only, MLP-only, and combined attention+MLP adapters, and examine the effects of rank, scaling factor, dropout, and learning rate. We conduct experiments on two tasks and find that LoRA achieves performance close to full fine-tuning while training far fewer parameters.

3 Related Work

Parameter-efficient fine-tuning methods have now emerged as an alternative to full fine-tuning for adapting large language models to downstream tasks. Among these approaches, Low-Rank Adaptation (LoRA) has become one of the most used techniques due to its simplicity and strong performance.

LoRA was introduced by Hu et al. (2022) [1] as a method for adapting pretrained transformers by learning low-rank updates to existing weight matrices while keeping the original parameters frozen. A pretrained weight matrix is modified through a low-rank decomposition. LoRA lets models adapt to new tasks while training a small fraction of the parameters through this restriction of a low-dimensional subspace. This original LoRA paper focuses primarily on adapting attention projection matrices and found that adapting the query and value projections often produced strong performance. Moreover, the authors did not extensively evaluate adaptation of feed-forward MLP layers. This leaves the question of optimal adapter placement largely open.

Subsequent work has explored LoRA hyperparameters and adapter placement in greater detail. QLoRA: Efficient Finetuning of Quantized LLMs by Tim Dettmers et al. (2023) [2] performs large-scale hyperparameter sweeps across rank, dropout, and adapter placement. Their experiments suggest that when adapters are applied broadly across the network, performance can become less sensitive to rank, revealing that adapter coverage may be more important than just rank. They also report that moderate dropout values can improve performance in smaller models.

Several studies further investigate where LoRA adapters should be applied within the transformer architecture. LoRA Learns Less and Forgets Less by Stella Biderman et al. (2024) [3] reports that adapting MLP layers or all transformer modules often outperform attention-only adaptation, suggesting that the feed-forward layers have a key role in task-specific adaptation.

Other work has investigated the interaction between LoRA hyperparameters. For example, LoRA+: Efficient Low Rank Adaptation of Large Models [4] analyzes the optimization dynamics of LoRA and demonstrates that the two low-rank factors may benefit from different learning rates.

More recent LoRA-specific work has also explored how the method behaves across different settings and tasks. DoRA: Weight-Decomposed Low-Rank Adaptation [5] revisits the gap between LoRA

and full fine-tuning and reports improvements on tasks such as image understanding. HydraLoRA: An Asymmetric LoRA Architecture for Efficient Fine-Tuning [6] proposes an asymmetric LoRA architecture that improves parameter efficiency and downstream performance. In the multi-task setting, MTL-LoRA: Low-Rank Adaptation for Multi-Task Learning [7] shows that standard LoRA can struggle with task interference, and proposes a modified design that improves results across natural language understanding tasks.

Together, these studies highlight several open questions about LoRA design choices, including where adapters should be inserted, how rank affects performance, and how hyperparameters such as α , dropout, and learning rate should be tuned. While the original LoRA work focused primarily on attention projections, later studies suggest that broader adaptation may improve performance depending on the task.

These findings directly motivate our experimental design. Following the suggestion in Biderman et al. [3] that MLP adaptation may be underexplored, we include MLP-only as a first-class configuration alongside attention-only variants. Motivated by the QLoRA [2] finding that adapter coverage may matter more than rank, we test configurations across multiple placement strategies before tuning rank. Finally, informed by LoRA+ [4], we include learning rate variation in our hyperparameter sweeps, though we leave per-factor learning rate schedules to future work.

4 Approach

Our project investigates parameter-efficient adaptation of large language models using Low-Rank Adaptation (LoRA). Rather than fine-tuning all parameters of a pretrained language model, LoRA introduces low-rank updates to existing weight matrices while keeping the original parameters frozen. This reduces the number of trainable parameters while still allowing the model to adapt to downstream tasks.

Following [1], the adapted weight matrix is written as

$$W = W_0 + \frac{\alpha}{r}BA,$$

where W_0 is the frozen pretrained weight matrix and $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d \times r}$ are trainable low-rank matrices. The rank $r \ll d$ constrains the update to a low-dimensional subspace, allowing the model to adapt using far fewer parameters than full fine-tuning.

We implement LoRA directly within our GPT-2 model. Our implementation allows LoRA to be applied to several different components of the model:

- attention projection matrices (query, value, key, and output)
- feed-forward MLP layers
- embedding layers

Following [1], we used a random Gaussian initialization for A and zero for B . Common alternate approaches, like the Hugging Face PEFT library, use a Kaiming uniform initialization for A [8]. This could explain the performance differences between the Hugging Face library and our implementation.

This flexibility allows us to systematically study where LoRA should be applied within the transformer architecture. In particular, the original LoRA work focused primarily on adapting attention projections. However, later work suggests that adapting other components may improve performance depending on the task.

Our experiments therefore evaluate several adaptation strategies:

- attention-only LoRA, adapting the attention projections
- MLP-only LoRA, adapting the feed-forward layers
- combined attention + MLP adaption
- full LoRA, attention + MLP + embedding adaptation

We evaluate these configurations across two tasks:

1. Paraphrase detection, a classification task based on semantic similarity
2. Sonnet generation, a generative task requiring stylistic text production

These tasks represent two different adaptation regimes: structured classification versus creative generation. By comparing LoRA configurations across both tasks, we aim to understand how parameter-efficient adaptation behaves under different learning objectives.

Our main baseline is the standard full fine-tuning setup from the default project, in which all GPT-2 parameters are updated for the downstream task. We compare our results against this baseline to evaluate whether LoRA can match strong performance while training far fewer parameters.

5 Experiments

5.1 Data

We evaluate our models on two tasks.

Paraphrase Detection

For classification experiments we use the Quora Question Pairs (QQP) dataset, as described in the CS224N Default Final Project instructions. Each example contains a pair of questions and a binary label indicating whether they express the same meaning.

The task is framed as generative classification: the GPT-2 model receives both questions and generates either “yes” or “no”. The dataset is split into training, development, and test sets, and we use classification accuracy on the development set to evaluate performance.

Sonnet Generation

To evaluate LoRA on a generative task, we train GPT-2 on a corpus of Shakespearean sonnets. The objective is to generate stylistically consistent sonnets that match the reference text.

This task tests whether low-rank adaptation can capture stylistic and structural properties of poetic language while modifying only a small number of model parameters.

5.2 Evaluation method

For paraphrase detection, we evaluate model performance using accuracy, defined as the proportion of correctly classified sentence pairs in the development set.

For sonnet generation, we evaluate model outputs using chrF score, a character-level similarity metric that measures how closely generated text matches reference text. This metric is well suited for evaluating poetic text, where small variations in wording and spelling may still preserve stylistic structure.

5.3 Experimental details

We evaluate multiple adapter placements described in Section 4, including attention-only (qv, qvko), MLP-only, MLP+attention, and full adaptation including embeddings.

Across experiments we vary several LoRA design choices: rank $r \in \{4, 8, 16\}$, scaling factor $\alpha \in \{4, 8, 16, 32\}$, dropout $p \in [0, 0.3]$. Learning rates were primarily set to 10^{-5} , with additional runs using 5×10^{-6} , 2×10^{-5} , 3×10^{-5} , and 10^{-4} for exploratory experiments. Most configurations were trained up to 20 epochs with early stopping based on development set performance.

Due to computational constraints, each configuration was trained using a single run rather than multiple random seeds. Consequently, the reported metrics represent single-run results and may include some variance from training stochasticity. While the main performance trends were consistent across experiments, small differences between configurations should be interpreted cautiously.

5.4 Results

Paraphrase Detection

On the test set leaderboard, paraphrase detection achieves a score of 0.873.

Full fine-tuning achieves the highest performance on the QQP development set with an accuracy of 0.893.

When comparing our LoRA implementation to Hugging Face’s PEFT library with a configuration of adapting the query and value matrices, rank $r = 4$, scaling factor $\alpha = 4$, and dropout $p = 0$, we found that our implementation results in an accuracy of 0.854 compared to the library accuracy of 0.823.

Table 2 reports the development accuracy across LoRA configurations. The best LoRA configuration applies adapters to both MLP and attention layers with rank $r = 8$, scaling factor $\alpha = 16$, and dropout $p = 0$. This configuration achieves a development accuracy of 0.888 while training 1.33M parameters, substantially fewer than the 124M updated during full fine-tuning.

Overall, these results demonstrate that LoRA can achieve performance close to full fine-tuning while training only a small fraction of model parameters.

Figure 1 compares adapter placement strategies under a controlled configuration ($r = 4$, $\alpha = 4$, learning rate 10^{-5} , no dropout). Development accuracy increases from 0.854 for qv adapters to 0.860 for MLP-only adapters, and reaches 0.873 when both MLP and attention layers are adapted.

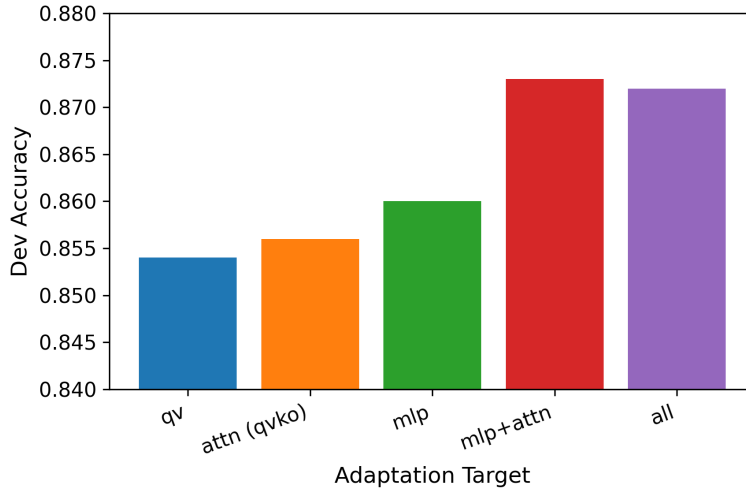


Figure 1: QQP dev accuracy for different LoRA adaptation targets under a controlled configuration ($r = 4$, $\alpha = 4$, learning rate 10^{-5} , no dropout).

Increasing rank from $r = 4$ to $r = 8$ improved performance in several configurations, though gains were modest and not consistent across all settings. Scaling factors between $\alpha = 8$ and $\alpha = 16$ performed best in our experiments, while larger values such as $\alpha = 32$ slightly degraded performance. Dropout showed no consistent benefit; while runs with $p = 0.05$ achieved similar accuracy, the best configuration used no dropout.

Learning rate showed minimal variation in our experiments, and most runs, including the best run, used 10^{-5} . In terms of training time, the best LoRA model required 39,017 seconds, while smaller configurations were noticeably faster but produced lower accuracy. These results show that LoRA approaches the performance of full fine-tuning on QQP while updating only about 1% of the model parameters.

Sonnet Generation

On the test set leaderboard, sonnet generation achieves a chrF score of 40.637.

When comparing our LoRA implementation to our Hugging Face PEFT library implementation with a configuration of adapting the query and value matrices, rank $r = 4$, scaling factor $\alpha = 4$, and dropout $p = 0$, we found that our implementation results in an accuracy of 39.440 compared to the library accuracy of 40.170.

Table 7 reports the development accuracy across LoRA configurations. The best configuration adapts all transformer layers except embeddings, with rank $r=8$, scaling factor $\alpha = 32$, learning rate 2×10^{-5} , and dropout 0.20. This configuration achieves a chrF score of 41.94 on the development set, which outperformed the full fine-tuned GPT-2 baseline of 40.417. It only trained 1.33 million parameters, much fewer than the full 124 million.

Adapting both MLP and attention layers produced the strongest generation results among the tested configurations. The best MLP-only run achieved 40.850, while attention-only reached 41.107, and adapting multiple modules achieved 41.94.

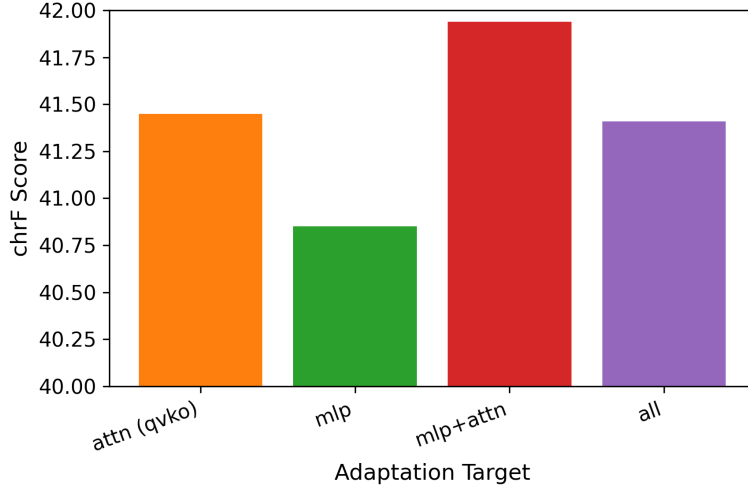


Figure 2: Dev chrF for different LoRA adaptation targets under a controlled configuration ($r = 8$, $\alpha = 32$, learning rate 2×10^{-5} , dropout $p = 0.2$). Adapting both the MLP and attention projections yields the strongest performance among the LoRA variants.

In addition, adjusting hyperparameters brought about large changes. First, we observed the rank’s relationship with adapter placement. With the attention-only runs, increasing rank was helpful and chrF rose from rank $r = 4$ to $r = 32$. However, when LoRA was applied broadly, larger rank was no longer clearly beneficial. In the all-except-embed runs, rank $r = 8$ produced the best result, with rank $r = 16$ and $r = 32$ performing worse. So, a larger rank is not uniformly better and its helpfulness depends on where LoRA is applied.

When considering the LoRA configuration with both MLP and attention layers, increasing the scaling factor from $\alpha = 4$ to $\alpha = 32$ greatly improved the chrF, from 38.136 to 41.277. Moreover, doubling the learning rate from 10^{-5} to 2×10^{-5} increased the chrF. Dropout also showed a similar trend, with $p = 0.2$ producing the best performance.

Training time results show the expected tradeoff between efficiency and performance. Our best model required 486.2s. Narrower LoRA settings were faster, such as mlp-only at $r = 4$ taking 260.5s and attention-only at $r = 4$ taking 316.8s, but these produced a lower chrF.

6 Analysis

Our experiments highlight several important trends in low-rank adaptation.

Adapter placement

Adapter placement plays a major role in LoRA performance. Across both tasks, configurations that adapted both MLP and attention layers achieved the strongest results. MLP-only adapters performed better than attention-only adapters on paraphrase detection, suggesting that feed-forward layers substantially contribute to the representations used for semantic classification. When both modules are adapted simultaneously, the model is able to modify both attention patterns and intermediate feature transformations, allowing for stronger task adaptation without needing to update the full parameter set.

For sonnet generation, broader adapter placement improved performance compared to narrower attention-only configurations. This suggests that stylistic generation tasks need deeper changes to the model’s internal representations, which may involve both attention mechanisms and feed-forward transformations.

In both tasks, we observe that embedding adaptation does not provide much benefit relative to its additional parameter cost.

Rank Scaling

Increasing the rank from $r = 4$ to $r = 8$ improved performance in most configurations, but higher ranks showed diminishing or negative returns, particularly when adapters were applied broadly. For sonnet generation, attention-only configurations showed continued improvement up to $r = 32$, while the combined MLP+attention configuration peaked at $r = 8$, with $r = 16$ and $r = 32$ performing worse (Table 7). This interaction between rank and adapter placement suggests that broader module coverage reduces the effective dimensionality needed per adapter: when more modules participate in adaptation, each module requires fewer degrees of freedom. This is consistent with the QLoRA finding [2] that adapter coverage can compensate for lower rank.

The scaling factor α

The optimal scaling parameter differed between tasks. For paraphrase detection, performance peaked at $\alpha = 16$, while generation performance improved when $\alpha = 32$.

This difference likely reflects the different adaptation requirements of the two tasks. One possible explanation is that classification requires relatively small adjustments to existing representations, while generative tasks benefit from stronger updates to capture stylistic features.

Dropout effects

Dropout had mixed effects depending on the task. For paraphrase detection, moderate dropout values (e.g., $p = 0.05$) sometimes reduced overfitting but did not consistently improve final accuracy. In contrast, sonnet generation benefited from larger dropout values, with the best results occurring at $p = 0.2$.

Parameter efficiency

Our results also highlight the parameter efficiency of LoRA-based adaptation. The strongest LoRA configuration trains approximately 1.33M parameters, representing roughly 1% of the total 124M GPT-2 parameters, yet achieves development accuracy within 0.6% of full fine-tuning on the QQP task. This suggests that low-rank updates can capture much of the task-specific adaptation required for both classification and generative tasks while updating only a small fraction of model parameters.

We also observe that adapter placement influences the tradeoff between parameter count and efficiency. MLP-only adapters achieve strong performance with fewer trainable parameters, while combining attention and MLP adapters yields slightly better performance at a reasonable increase in parameter count.

Together, these results demonstrate that LoRA can significantly reduce training cost while preserving strong performance, but that careful hyperparameter tuning is necessary to achieve the best results.

Limitations

Our results are based on single training runs per configuration, so small performance differences (e.g., the 0.001 gap between 0.881 and 0.882 in Table 2) may reflect stochastic variation rather than meaningful trends. We focus our analysis on the larger, more consistent patterns—such as the ~2 point improvement in QQP accuracy from attention-only to MLP+attention placement—which are less likely to be artifacts of randomness. Additionally, chrF evaluates character-level similarity to reference text and does not directly capture poetic properties such as meter, rhyme scheme, or thematic coherence. A human evaluation or more targeted metrics would provide a more complete picture of generation quality. Finally, our experiments are limited to GPT-2 (124M parameters); the optimal LoRA configurations may differ for larger models where the low-rank hypothesis may hold more strongly.

7 Conclusion

Our experiments demonstrate that LoRA provides a practical path to parameter-efficient adaptation of GPT-2 across both classification and generation tasks. On paraphrase detection, LoRA achieves 0.888 development accuracy—within 0.6% of full fine-tuning—while training roughly 1% of the model’s parameters. On sonnet generation, LoRA not only matches but exceeds the full fine-tuning baseline, achieving 41.94 chrF versus 40.417. This result suggests that constraining updates to a low-rank subspace can act as a beneficial regularizer for generative tasks, preventing the model from overfitting to a small training corpus.

Several design choices proved important. Adapting both MLP and attention layers generally produced the strongest results. Small ranks such as $r = 8$ were sufficient to achieve strong performance our experiments. Finally, the optimal scaling parameter and dropout rate depended on the task, indicating that LoRA hyperparameters must be tuned carefully for different learning objectives.

Overall, our experiments demonstrate that LoRA provides a practical and effective approach for parameter-efficient fine-tuning of transformer models. Future work should evaluate each configuration across multiple random seeds to better quantify variance and confirm the robustness of these trends, and could explore adaptive rank selection, dynamic scaling strategies, or combining LoRA with other parameter-efficient techniques such as quantization or prefix tuning.

Team contributions (Required for multi-person team)

Joseph completed the GPT-2 implementation and paraphrase detection implementation and training. Maya completed the sentiment analysis portion and sonnet generation implementation and training.

References

- [1] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [3] Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John Patrick Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024. Featured Certification.
- [4] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. LoRA+: Efficient low rank adaptation of large models. In *Forty-first International Conference on Machine Learning*, 2024.
- [5] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.

- [6] Chunlin Tian, Zhan Shi, Zhijiang Guo, Li Li, and Cheng zhong Xu. HydraloRA: An asymmetric loRA architecture for efficient fine-tuning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [7] Yaming Yang, Dilxat Muhtar, Yelong Shen, Yuefeng Zhan, Jianfeng Liu, Yujing Wang, Hao Sun, Weiwei Deng, Feng Sun, Qi Zhang, Weizhu Chen, and Yunhai Tong. Mtl-lora: low-rank adaptation for multi-task learning. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’25/IAAI’25/EAAI’25. AAAI Press, 2025.
- [8] Hugging Face. Lora. https://huggingface.co/docs/peft/main/en/conceptual_guides/lora, 2026. PEFT documentation.

A Appendix

A.1 Hyperparameter search results

Method	Dev Acc	Δ vs FT	Trainable Params (M)
Full Fine-Tuning	0.893	–	124
LoRA (qv)	0.854	-0.039	0.15
LoRA (qvko)	0.856	-0.037	0.30
LoRA (MLP)	0.860	-0.033	0.37
LoRA (MLP+Attn)	0.873	-0.020	0.67
LoRA (All)	0.872	-0.021	0.88

Table 1: Adapter placement comparison on QQP. Full fine-tuning provides the strongest performance but requires updating the entire GPT-2 model (124M parameters). LoRA adapters train fewer than 1M parameters while achieving comparable accuracy. Adapting both feed-forward and attention layers yields the strongest LoRA performance. Each configuration has $r = 4$, $\alpha = 4$, $\text{lr} = 10^{-5}$, and no dropout.

Target	r	α	p	Dev Acc
qvko	8	16	0.05	0.877
mlp	4	16	0.05	0.859
mlp	8	16	0	0.881
mlp	8	16	0.05	0.882
mlp	8	32	0.05	0.879
mlp+attn	8	16	0	0.889
mlp+attn	8	16	0.05	0.887

Table 2: Selected QQP hyperparameter comparisons. We compare attention-only, MLP-only, and MLP+attention LoRA placements under closely matched settings. Moving from attention-only to MLP-based adaptation improves accuracy, and increasing rank from $r = 4$ to $r = 8$ improves MLP performance. Dropout $p = 0.05$ provides a small gain for MLP-only adapters but does not improve the best MLP+attention configuration. The best performing learning rate was found to be 10^{-5} , which was used for the above tests.

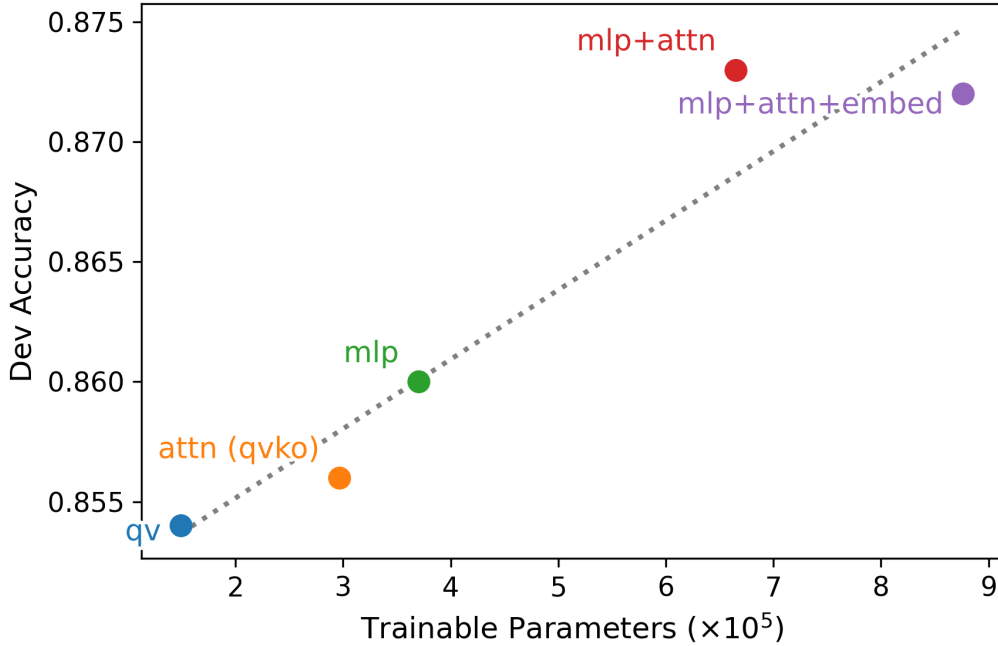


Figure 3: Parameter efficiency of LoRA configurations on QQP. Each point represents a different adaptation target, plotted against the number of trainable parameters (in units of 10^5). Each configuration has $r = 4$, $\alpha = 4$, $\text{lr} = 1 \times 10^{-5}$, and no dropout. While larger adapter placements improve accuracy, they require more trainable parameters. The dotted line shows a linear trend illustrating the trade-off between parameter count and performance.

Method	chrF	Params (M)
LoRA (qvko)	41.45	0.59
LoRA (MLP)	40.85	0.74
LoRA (MLP+Attn)	41.94	1.33
LoRA (All)	41.41	1.75

Table 3: Effect of adapter placement on sonnet generation. The full fine-tuned GPT-2 baseline (124M params) achieves chrF = 40.417, while LoRA configurations train between 0.59M and 1.75M parameters. Broader adapter placement improves generation quality, with configurations adapting both attention and MLP layers outperforming attention-only adapters. Each configuration has $r = 8$, $\alpha = 32$, $\text{lr} = 2 \times 10^{-5}$, and $p = 0.2$.

Target	r	α	LR	p	chrF
mlp+attn	8	32	1×10^{-5}	0.10	41.43
mlp+attn	8	32	2×10^{-5}	0.10	41.50
mlp+attn	8	32	2×10^{-5}	0.20	41.94
mlp+attn	8	32	2×10^{-5}	0.25	41.52
mlp+attn	8	32	2×10^{-5}	0.30	41.38

Table 4: Hyperparameter sweep for sonnet generation using MLP+attention adapters. Moderate dropout ($p \approx 0.2$) and learning rate 2×10^{-5} produce the best generation quality.

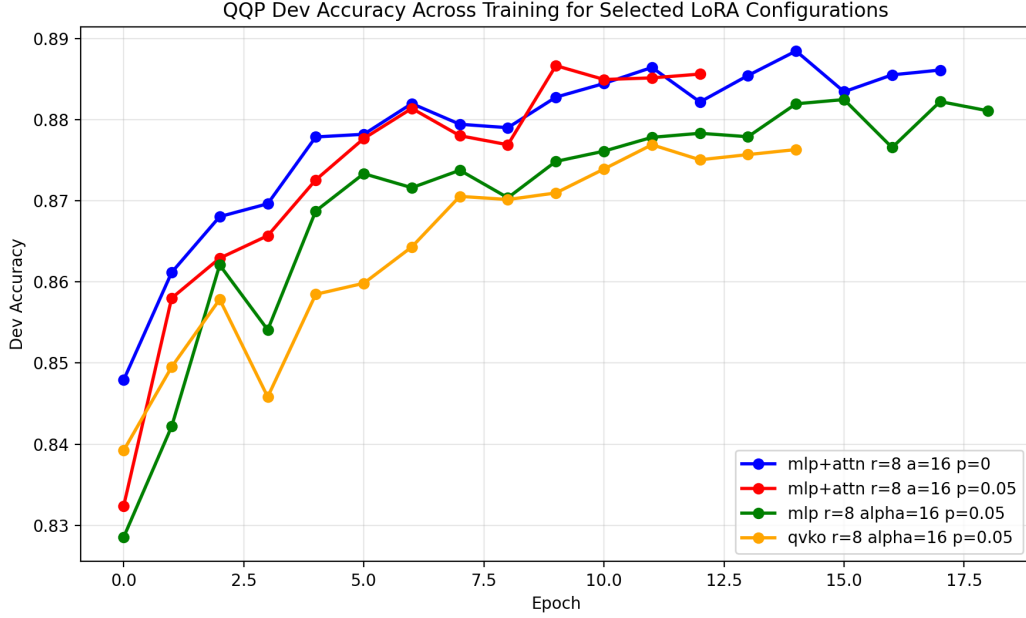


Figure 4: QQP development accuracy across training epochs for selected LoRA configurations. All configurations use rank $r = 8$ and scaling factor $\alpha = 16$ with learning rate 10^{-5} . The two MLP+attention configurations (blue, red) consistently outperform MLP-only (green) and attention-only (orange) adapters, with the performance gap emerging by epoch 3–4 and persisting throughout training. The MLP+attention configuration with dropout $p = 0.05$ (red) converges to its peak earlier and triggers early stopping around epoch 10, while the no-dropout variant (blue) continues improving gradually through epoch 15. Both single-module configurations plateau at a lower accuracy band, reinforcing the finding that broader adapter placement improves classification performance.

Target	r	α	LR	p	chrF
mlp+attn	8	4	10^{-5}	0.10	38.14
mlp+attn	8	8	10^{-5}	0.10	39.25
mlp+attn	8	16	10^{-5}	0.10	40.53
mlp+attn	8	32	10^{-5}	0.10	41.43

Table 5: Effect of LoRA scaling parameter α on sonnet generation for MLP+attention adapters. Holding the adapter target, rank, learning rate, and dropout fixed ($r = 8$, $lr = 10^{-5}$, $p = 0.1$), chrF increases monotonically as α increases from 4 to 32. Across all sonnet generation experiments, higher values of α tend to achieve higher chrF scores.

Table 6: LoRA hyperparameter search for paraphrase detection

target	r	α	lr	p	dev acc	best epoch	params (M)	avg tok/s	training time (s)
qv	4	4	1e-5	0	0.842	10	0.15	—	—
qv	4	4	1e-5	0	0.854	20	0.15	8502.200	3.2e+04
qvko	4	4	1e-5	0	0.856	10	0.30	6210.700	2.2e+04
mlp	4	4	1e-5	0	0.860	10	0.37	8140.800	1.7e+04
mlp+attn	4	4	1e-5	0	0.873	10	0.67	5468.100	2.5e+04
all	4	4	1e-5	0	0.872	9	0.88	4895.700	2.8e+04
qv	4	8	1e-5	0	0.851	10	0.15	8575.600	1.6e+04
qv	4	16	1e-5	0	0.857	10	0.15	8646.600	1.6e+04
mlp+attn	8	16	1e-5	0	0.888	15	1.33	6268.900	3.9e+04
mlp	4	32	1e-4	0.1	0.758	4	0.37	8825.300	9.2e+03
mlp	4	32	3e-5	0.1	0.827	4	0.37	6966.100	1.2e+04
mlp	4	16	1e-5	0.05	0.859	4	0.37	7718.100	1.1e+04
mlp	8	16	1e-5	0.05	0.882	16	0.74	8007.800	3.2e+04
mlp	8	16	1e-5	0	0.881	11	0.74	8480.100	2.2e+04
mlp	8	16	5e-6	0.05	0.881	20	0.74	7136.600	3.8e+04
mlp+attn	8	16	1e-5	0.05	0.887	10	1.33	6241.600	2.8e+04
mlp	8	16	1e-5	0.1	0.874	8	0.74	8840.700	1.7e+04
qvko	8	16	1e-5	0.05	0.877	12	0.59	7670.100	2.7e+04
mlp	8	32	1e-5	0.05	0.879	11	0.74	7345.900	2.6e+04
mlp	8	8	1e-5	0.05	0.881	20	0.74	7137.600	3.8e+04

Table 7: LoRA hyperparameter search for sonnet generation

target	r	α	lr	p	chrF	best epoch	params (M)	avg tok/s	training time (s)
mlp	4	32	1e-5	0	40.466	7	0.37	804.400	2.6e+02
qvko	4	32	1e-5	0	40.607	7	0.29	661.500	3.2e+02
qvko	4	32	1e-5	0	40.813	12	0.29	610.700	5.1e+02
qvko	8	32	1e-5	0	39.989	6	0.59	659.400	3.2e+02
qvko	16	32	1e-5	0	40.247	6	1.18	643.700	3.3e+02
qvko	32	32	1e-5	0	41.107	8	2.36	641.000	3.3e+02
qvko	4	32	1e-5	0.1	40.745	6	0.29	551.200	3.8e+02
mlp+attn	4	32	1e-5	0	41.270	8	0.66	562.500	3.7e+02
mlp+attn	4	32	1e-5	0.1	41.294	5	0.66	435.100	4.8e+02
mlp+attn	8	32	1e-5	0.1	41.427	9	1.33	522.300	4e+02
mlp+attn	16	32	1e-5	0.1	41.181	8	2.65	508.100	4.1e+02
mlp+attn	8	32	1e-5	0.05	41.277	9	1.33	493.500	4.2e+02
mlp+attn	8	4	1e-5	0.05	38.136	1	1.33	606.800	3.5e+02
mlp+attn	8	8	1e-5	0.1	39.251	8	1.33	637.200	3.3e+02
mlp+attn	8	16	1e-5	0.1	40.528	5	1.33	521.800	4e+02
mlp+attn	8	16	1e-5	0.2	40.595	9	1.33	388.400	5.4e+02
mlp+attn	16	16	1e-5	0.1	40.389	8	2.65	529.000	4e+02
mlp+attn	8	32	2e-5	0.1	41.500	3	1.33	445.300	4.7e+02
mlp+attn	8	32	1e-4	0.1	41.342	8	1.33	443.400	4.7e+02
mlp+attn	8	32	2e-5	0.2	41.938	8	1.33	431.000	4.9e+02
mlp+attn	8	32	2e-5	0.3	41.378	4	1.33	358.200	5.8e+02
mlp+attn	8	32	2e-5	0.25	41.524	7	1.33	349.300	6e+02
mlp+attn	8	32	3e-5	0.2	41.174	4	1.33	305.800	6.9e+02
mlp	8	32	2e-5	0.2	40.850	7	0.74	441.900	4.7e+02
qvko	8	32	2e-5	0.2	41.449	8	0.59	360.300	5.8e+02
all	8	32	2e-5	0.2	41.414	9	1.75	378.200	5.5e+02
all	16	32	2e-5	0.2	41.229	3	3.50	303.300	6.9e+02
all	8	16	2e-5	0.2	41.142	9	1.75	331.000	6.3e+02
all	8	32	2e-5	0.3	41.587	9	1.75	307.500	6.8e+02
all	8	32	3e-5	0.3	41.716	8	1.75	278.300	7.5e+02
all	8	32	4e-5	0.3	41.648	7	1.75	287.900	7.3e+02
all	8	32	3e-5	0.4	41.263	4	1.75	294.700	7.1e+02